

# Velocity Obstacle for Polytopic Collision Avoidance for Distributed Multi-Robot Systems

Jihao Huang , Jun Zeng , Xuemin Chi, Koushil Sreenath , Zhitao Liu , *Member, IEEE*, and Hongye Su 

**Abstract**—Obstacle avoidance for multi-robot navigation with polytopic shapes is challenging. Existing works simplify the system dynamics or consider it as a convex or non-convex optimization problem with positive distance constraints between robots, which limits real-time performance and scalability. Additionally, generating collision-free behavior for polytopic-shaped robots is harder due to implicit and non-differentiable distance functions between polytopes. In this letter, we extend the concept of velocity obstacle (VO) principle for polytopic-shaped robots and propose a novel approach to construct the VO in the function of vertex coordinates and other robot's states. Compared with existing work about obstacle avoidance between polytopic-shaped robots, our approach is much more computationally efficient as the proposed approach for construction of VO between polytopes is optimization-free. Based on VO representation for polytopic shapes, we later propose a navigation approach for distributed multi-robot systems. We validate our proposed VO representation and navigation approach in multiple challenging scenarios including large-scale randomized tests, and our approach outperforms the state of art in many evaluation metrics, including completion rate, deadlock rate, and the average travel distance.

**Index Terms**—Distributed robot systems, collision avoidance, multi-robot systems.

## I. INTRODUCTION

### A. Motivation

THE multi-robot navigation is a challenging task [1], [2] that needs to be solved for various applications such as warehouse delivery and search and rescue operations [3]. The critical challenge of navigation tasks for the multi-robot system is to achieve real-time obstacle avoidance while navigating each robot to its respective destination, as shown in Fig. 1. Existing

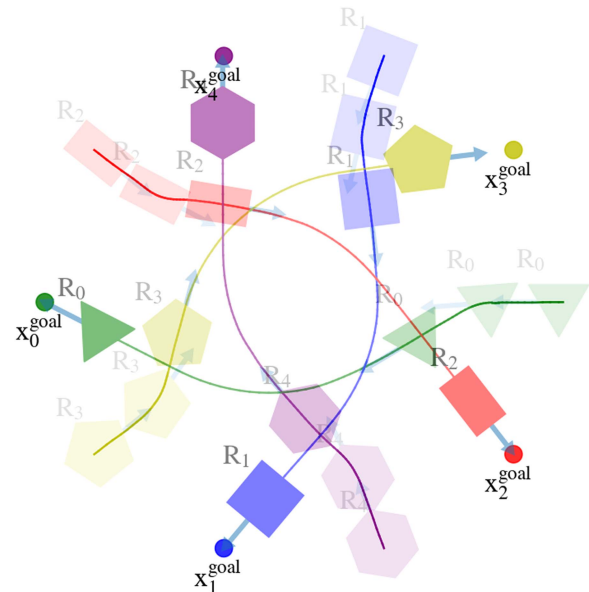


Fig. 1. Snapshot of the distributed multi-robot navigation with polytopic shapes using our proposed approach. Each robot  $R_i$  has its own destination  $x_i^{\text{goal}}$ , and each robot could decide independently to move towards the destination while avoiding collisions. In this figure, we demonstrate each robot position at four different ticks with lighter shades of a color indicating robot's position in the past.

approaches are not accessible to be deployed for polytopic multi-robot systems, as they usually consider the collision avoidance between polytopes as convex or even non-convex optimizations, whose computational complexity increases dramatically with the number of robots. Velocity obstacle [4], commonly abbreviated VO, is the set of all velocities of a robot that will result in a collision with an obstacle at some moment in time, assuming that the obstacle maintains its current velocity. In this letter, we propose a novel approach in the field of VO to achieve distributed multi-robot navigation with polytopic shapes, which could be deployed in real-time.

### B. Related Work

1) *Collision Avoidance Between Polytopes*: Collision avoidance between polytopes is the crucial approach to achieve non-conservative collision avoidance for polytopic-shaped robots. The polytopic shapes could be approximated into hyper-ellipses, however, this conservative over-approximation may lead to deadlock maneuvers, shown in [5]. Recently, optimization-based approaches have become popular in this field, where the obstacle avoidance criteria between polytopes are added

Manuscript received 31 January 2023; accepted 4 April 2023. Date of publication 21 April 2023; date of current version 28 April 2023. This letter was recommended for publication by Associate Editor G. Pereira and Editor M. Ani Hsieh upon evaluation of the reviewers' comments. This work was supported in part by the National Key R&D Program of China under Grant 2021YFB3301000, in part by the National Natural Science Foundation of China (NSFC) under Grant 62173297, in part by the Zhejiang Key R&D Program under Grant 2022C01035, and in part by the Fundamental Research Funds for the Central Universities under Grant 226-2022-00086. (Jihao Huang and Jun Zeng contributed equally to this work.) (Corresponding author: Zhitao Liu.)

Jihao Huang, Xuemin Chi, Zhitao Liu, and Hongye Su are with the State Key Laboratory of Industrial Control Technology, Institute of Cyber-Systems and Control, Zhejiang University, Hangzhou 310027, China (e-mail: jiahoah@zju.edu.cn; chixuemin@zju.edu.cn; ztliu@zju.edu.cn; hysu69@zju.edu.cn).

Jun Zeng and Koushil Sreenath are with the Hybrid Robotics Group at the Department of Mechanical Engineering, UC Berkeley, Berkeley, CA 94709 USA (e-mail: zengjunsju@berkeley.edu; koushils@berkeley.edu).

This letter has supplementary downloadable material available at <https://doi.org/10.1109/LRA.2023.3269295>, provided by the authors.

Digital Object Identifier 10.1109/LRA.2023.3269295

as constraints. However, the design of obstacle avoidance constraint is challenging since the distance function between two polytopes is also an optimization problem — it is implicit and does not hold an analytical expression [6]. Authors in [7] propose an optimization-based approach that keeps all the vertices of the controlled target outside all the other obstacles to achieve collision avoidance between rectangle-shaped robots, but this approach is neither applicable for the online calculation nor for the general polytopic shapes, especially in a densely packed environment. To deal with obstacle avoidance between general polytopic shapes, mixed-integer programming [8] applies well for collision avoidance between two polytopic-shaped robots with linear dynamics, but it cannot be deployed as a real-time controller or trajectory planner for general nonlinear dynamical systems due to the added complexity from the nonlinear mixed-integer optimization problem. A duality-based approach [9], [10] reformulates the implicit constraints of collision avoidance into smooth, differentiable constraints, which optimizes the computational complexity to some extent, but it is unsuitable for online planning of nonlinear systems. This methodology has been extended into a distributed multi-robot system [11] which achieves real-time path planning for nonlinear systems based on a bi-level optimization scheme. The dual approach has also been generalized for general nonlinear continuous-time [12] and discrete-time [13] systems to achieve real-time collision avoidance through convex [12] or non-convex [13] optimizations with control barrier functions [14], [15], but not in a distributed manner. Notice that all the optimization-based approaches mentioned above [9], [10], [11], [12], [13] usually suffer from a dramatic complexity increase with the number of the robots, which becomes challenging when deployed in real-time for a distributed multi-robot system. In this letter we propose an optimization-free approach to address the above issues.

2) *Velocity Obstacle for Distributed Multi-Robot Navigation:* Existing approaches for multi-robot navigation can be classified into centralized and distributed ones [16]. In contrast with centralized approaches, in distributed approaches each robot is an independent decision maker, and it could act independently according to the sensor information obtained from its onboard sensors [17]. Since distributed approaches do not require global centralized communication between robots, it becomes suitable for a large-scale multi-robot system to achieve navigation tasks with limited computational resources. One of its significant challenges is realizing reliable collision avoidance with limited sensor information and calculating optimal velocities with high efficiency and guaranteed safety.

Velocity obstacle (VO) and its variants have been widely used to realize local collision avoidance and navigation of the distributed multi-robot system in an open shared environment with static and dynamic obstacles. VO [4] is the set of all velocities of a robot that will result in a collision with an obstacle at some moment in time, assuming that the obstacle maintains its current velocity. So each robot of the distributed multi-robot system could select a velocity which is outside of any VO induced by the obstacles or other robots and close to the preferred velocity in each time step to achieve collision avoidance and navigation independently. Some variants of VO [18], [19], [20], [21], [22] make some improvements to reduce the unnecessary oscillations when using the VO-based algorithm for distributed multi-robot navigation, for more details readers can refer to Section II-C. Some other works [23], [24], [25] also take full advantage of VO and combine it with reinforcement learning (RL) to learn a

reliable collision avoidance policy for the distributed multi-robot navigation.

To summarize, VO and its variants are applicable for real-time navigation in distributed multi-robot systems, but current approaches are limited to circular robots and cannot be generalized to non-conservative avoidance of polytopic-shaped robots. This letter proposes an extension of the VO principle to polytopic-shaped robots and obstacles and uses it to solve a distributed multi-robot navigation task.

### C. Contributions

In this letter, we mainly focus on the problem of distributed multi-robot navigation with polytopic-shaped robots and obstacles. The contributions of this letter are as follows:

- We generalize the concept of VO to the polytopic-shaped robots and obstacles and propose a computationally efficient approach to construct the VO as a function of vertex coordinates and other robot's states.
- We propose a VO-based navigation approach for distributed multi-robot systems with polytopic shapes and validate it in many challenging scenarios, with multiple static and dynamic obstacles.
- We show that our proposed approach outperforms the state of the art in terms of completion rate, deadlock rate and the average travel distance through large-scale randomized tests.

### D. Organization

The rest of this letter is organized as follows: We formally define the problem of distributed multi-robot navigation with polytopic shapes, review the concept of VO and some variants of VO and describe the obstacle avoidance problem between polytopes in Section II. In Section III, we describe how to construct the VO for polytopic-shaped robots and demonstrate how to achieve the distributed multi-robot navigation with polytopic shapes using VO. We demonstrate our approach and present the simulation results in Section IV. Section V concludes the letter.

## II. PROBLEM STATEMENT & PRELIMINARIES

In this section, we firstly define the problem of distributed multi-robot navigation with polytopic shapes in Section II-A. Then, we present some preliminaries of velocity obstacle (VO), review two variants of VO (RVO and HRVO) and describe the obstacle avoidance problem between polytopes in Sections II-B, II-C and II-D, respectively.

### A. Problem Statement

Assume there are a set of  $N$  robots sharing an open space with a set of  $M$  static and dynamic non-reactive obstacles, i.e., their velocities are not adapted to avoid collision with the other robots. In this letter, we assume all robots and obstacles are polytopic-shaped. For notations, subscripts  $i$  and  $j$  are applied to distinguish robots and obstacles, where each robot is represented by  $R_i \in \mathbb{N} = \{R_0, R_1, \dots, R_{N-1}\}$  and each obstacle is represented by  $O_j \in \mathbb{O} = \{O_0, O_1, \dots, O_{M-1}\}$ , and subscript  $k$  is used to distinguish each vertex of a robot or an obstacle, i.e.,  $v_k$  represents the  $k$ th vertex. For each robot in a distributed multi-robot system, there are external and internal states and only the external ones could be observed by other robots through

TABLE I  
NOTATIONS AND DESCRIPTIONS

| Notation                                                            | Descriptions                                                                                                |
|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| $n$                                                                 | Dimension of the space                                                                                      |
| $N, M$                                                              | Numbers of robots and obstacles                                                                             |
| $\mathbb{N}, \mathbb{O}$                                            | Sets of robots and obstacles                                                                                |
| $R_i, O_j$                                                          | $i$ th robot and the $j$ th obstacle                                                                        |
| $r_{R_i}, r_{O_j}$                                                  | Radius of circular-shaped $R_i$ and $O_j$                                                                   |
| $\mathbf{x}_{R_i}, \mathbf{x}_{O_j}$                                | Current position coordinate of $R_i$ and $O_j$                                                              |
| $\mathbf{x}_i^{\text{goal}}$                                        | Goal position of $R_i$                                                                                      |
| $\mathbf{v}_{R_i}, \mathbf{v}_{O_j}$                                | Current velocity of $R_i$ and $O_j$                                                                         |
| $\mathbf{p}_{R_i}^k, \mathbf{p}_{O_j}^k$                            | $k$ th, $h$ th vertex coordinate of $R_i, O_j$                                                              |
| $v_{R_i, \text{max}}$                                               | Maximum velocity of $R_i$                                                                                   |
| $\text{CC}_{R_i O_j}$                                               | Collision cone for $R_i$ induced by $O_j$                                                                   |
| $\text{VO}_{R_i O_j}(\mathbf{v}_{O_j})$                             | Velocity obstacle for $R_i$ induced by $O_j$                                                                |
| $\text{RVO}_{R_i R_j}(\mathbf{v}_{R_i}, \mathbf{v}_{R_j})$          | Reciprocal VO for $R_i$ induced by $R_j$                                                                    |
| $\mathbf{v}_l, \mathbf{v}_r$                                        | Direction vectors on both sides of VO                                                                       |
| $S_i, A_i, b_i$                                                     | Set of polytopes                                                                                            |
| $\theta_{O_j}^{\mathbf{v}_{R_i}} - \theta_{R_i}^{\mathbf{v}_{O_j}}$ | Angle between vector $\mathbf{p}_{O_j}^{\mathbf{v}_{R_i}} - \mathbf{p}_{R_i}^{\mathbf{v}_{O_j}}$ and x axis |
| $\mathbf{e}_x, \mathbf{e}_y$                                        | Unit vectors of x and y axis                                                                                |
| $\Delta t$                                                          | Time step of simulation                                                                                     |
| $t_{\text{max}}$                                                    | Maximum loop time for the simulation                                                                        |
| $\mathbf{v}_{R_i}^{\text{pref}}, \mathbf{v}_{R_i}^{\text{new}}$     | Preferred or new velocity of $R_i$                                                                          |
| $l$                                                                 | Neighboring region of the robot                                                                             |
| $\mathbb{B}_i, \mathbb{C}_i$                                        | Neighboring robots and obstacles of the robot $R_i$                                                         |
| $\text{VO}_i$                                                       | Combined velocity obstacle of $R_i$                                                                         |
| $\text{RVO}_i$                                                      | Combined reciprocal velocity obstacle of $R_i$                                                              |
| $J(\mathbf{v}_{R_i})$                                               | Penalty cost function for velocity $\mathbf{v}_{R_i}$                                                       |
| $\phi_i$                                                            | Weight coefficient in the penalty function of $R_i$                                                         |
| $t_{c_i}(\mathbf{v}_{R_i})$                                         | Expected collision time when $R_i$ in velocity $\mathbf{v}_{R_i}$                                           |
| $v, w$                                                              | Transitional and rotational velocities                                                                      |
| $\text{VO}_c, \text{VO}_p$                                          | Circular-shaped or polytopic-shaped based VO                                                                |

the onboard sensors. The external states of robot  $R_i$  include the positions of all vertices  $\mathbf{p}_{R_i}^k$ , the current position  $\mathbf{x}_{R_i}$  and the current velocity  $\mathbf{v}_{R_i}$ . The robot's current position  $\mathbf{x}_{R_i}$  is a function of the position of its vertices, where  $\mathbf{x}_{R_i} = \frac{1}{K_{R_i}} \sum_{k=1}^{K_{R_i}} \mathbf{p}_{R_i}^k$  with the assumption that  $R_i$  has  $K_{R_i}$  vertices. The current velocity  $\mathbf{v}_{R_i}$  represents the time-derivative of the current position  $\mathbf{x}_{R_i}$ . The internal states include the goal position  $\mathbf{x}_i^{\text{goal}}$  and the maximum speed  $v_{R_i, \text{max}}$ . The states of the obstacle  $O_j$  include the current position  $\mathbf{x}_{O_j}$ , the current positions of all vertices  $\mathbf{p}_{O_j}^k$  and the current velocity  $\mathbf{v}_{O_j}$  (with  $\mathbf{v}_{O_j}$  being zero for static obstacles). All the states of obstacles are observable for all robots.

We define the distributed multi-robot navigation problem as follows: each robot needs to navigate to its goal position within the prescribed time while avoiding collision with other robots and obstacles independently. We list the notations for this letter in Table I for reference.

### B. Velocity Obstacle for Circular Shapes

In this section, we present some preliminaries of velocity obstacle representation [4]. For a circular-shaped robot  $R_i$  and a circular-shaped obstacle  $O_j$  with radius  $r_{R_i}$  and  $r_{O_j}$ , the current position and velocity could be denoted as  $\mathbf{x}_{R_i}, \mathbf{x}_{O_j}, \mathbf{v}_{R_i}, \mathbf{v}_{O_j}$  respectively. Before introducing the concept of VO, we first introduce the concept of collision cone (CC). For a robot  $R_i$  (with any shape) with velocity  $\mathbf{v}_{R_i}$  and an obstacle  $O_j$  with velocity  $\mathbf{v}_{O_j}$ ,  $\text{CC}_{R_i|O_j}$  is defined as follows [4]:

$$\text{CC}_{R_i|O_j} = \{\mathbf{v}_{R_i} - \mathbf{v}_{O_j} \mid \lambda(\mathbf{x}_{R_i}, \mathbf{v}_{R_i} - \mathbf{v}_{O_j}) \cap O_j \oplus -R_i \neq \emptyset\}$$

where  $\lambda(\mathbf{x}, \mathbf{v}) = \{\mathbf{x} + t\mathbf{v} \mid t > 0\}$  denotes a ray starting at point  $\mathbf{x}$  and in the direction of vector  $\mathbf{v}$ , and  $\oplus$  denotes the Minkowski

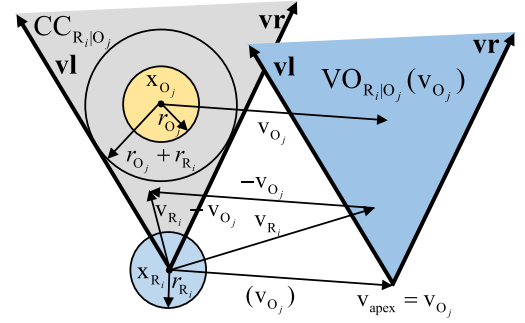


Fig. 2. Circular-shaped based velocity obstacle  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$  (blue conic region) of robot  $R_i$  (blue circular region) induced by the obstacle  $O_j$  (yellow circular region) with velocity  $\mathbf{v}_{O_j}$ .  $\text{CC}_{R_i|O_j}$  (gray conic region) represents the collision cone between  $R_i$  and  $O_j$ . If the relative velocity  $\mathbf{v}_{R_i} - \mathbf{v}_{O_j} \in \text{CC}_{R_i|O_j}$  or the absolute robot velocity  $\mathbf{v}_{R_i} \in \text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$ , a collision will occur between  $R_i$  and  $O_j$ . The direction vectors  $\mathbf{v}_l$  and  $\mathbf{v}_r$  (bold solid lines) is same for  $\text{CC}_{R_i|O_j}$  and  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$  of robot  $R_i$ .

sum,  $O_j \oplus -R_i$  means considering  $R_i$  as a point with a certain expansion for  $O_j$ . Thus, CC is the set of all relative velocities which lead to a collision. In particular, if the relative velocity of  $R_i$  and  $O_j$  lies in  $\text{CC}_{R_i|O_j}$ , and both  $R_i$  and  $O_j$  maintain the current velocity, a collision will occur between  $R_i$  and  $O_j$  in the future. Therefore, any relative velocity outside  $\text{CC}_{R_i|O_j}$  is guaranteed to be collision free, provided both  $R_i$  and  $O_j$  maintain the current velocity. The term  $\text{CC}_{R_i|O_j}$  is defined in terms of relative velocity. When considering multiple obstacles, it is necessary to establish an equivalent description that uses the absolute velocity of  $R_i$ . Adding the relative velocity  $\mathbf{v}_{R_i} - \mathbf{v}_{O_j}$  to  $\mathbf{v}_{O_j}$ , we can define the convex region  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j}) = \text{CC}_{R_i|O_j} \oplus \mathbf{v}_{O_j}$ , that is

$$\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j}) = \{\mathbf{v}_{R_i} \mid \lambda(\mathbf{x}_{R_i}, \mathbf{v}_{R_i} - \mathbf{v}_{O_j}) \cap O_j \oplus -R_i \neq \emptyset\} \quad (1)$$

as shown in Fig. 2. If  $R_i$  holds a constant velocity  $\mathbf{v}_{R_i} \in \text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$ , a collision will occur between  $R_i$  and  $O_j$  in the future, and vice versa.

However, constructing the VO through (1) is often with high computational complexity because it needs extensive sampling and judgment [4]. In fact, VO defines a geometric conic region of infeasible velocities for the robot. We can use  $\mathbf{v}_l$  and  $\mathbf{v}_r$  to the left and the right side of VO and  $\mathbf{v}_{\text{apex}}$  to represent the apex of the conic region in order to define VO [25], as shown in Fig. 2. Moreover, since  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$  is translated from  $\text{CC}_{R_i|O_j}$ , the  $\mathbf{v}_l$  and  $\mathbf{v}_r$  of  $\text{CC}_{R_i|O_j}$  is same as for  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$  and only the apex is different. The apex  $\mathbf{v}_{\text{apex}}$  of  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$  is at  $\mathbf{v}_{O_j}$ , as shown in Fig. 2. So if we could calculate the direction vectors  $\mathbf{v}_l$  and  $\mathbf{v}_r$  on both sides of  $\text{CC}_{R_i|O_j}$ , we can construct  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$  easily. Most of VO-based works [18], [19], [20] usually choose circular-shaped robots and obstacles, then the direction vectors  $\mathbf{v}_l$  and  $\mathbf{v}_r$  on both sides of  $\text{CC}_{R_i|O_j}$  can be conveniently obtained by calculating the vectors which are tangent to  $D(\mathbf{x}_{O_j} - \mathbf{x}_{R_i}, r_{R_i} + r_{O_j})$ , where  $D(\mathbf{x}, r)$  is a circle with radius  $r$  centered at  $\mathbf{x}$ . Then we can construct  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$  with the direction vectors  $\mathbf{v}_l$  and  $\mathbf{v}_r$  and the apex. In this case, the definition (1) of VO could be simplified to

$$\begin{aligned} \text{VO}_{R_i|O_j}(\mathbf{v}_{O_j}) \\ = \{\mathbf{v}_{R_i} \mid \exists t > 0, (\mathbf{v}_{R_i} - \mathbf{v}_{O_j})t \in D(\mathbf{x}_{O_j} - \mathbf{x}_{R_i}, r_{R_i} + r_{O_j})\}. \end{aligned}$$

However, the above approach is no longer valid when applied to polytopic-shaped robots, so in this letter we propose constructing the VO for polytopic-shaped robots through an alternative numerically efficient way instead of (1).

*Remark 1:* In fact,  $\mathbf{vl}$  and  $\mathbf{vr}$  represent the left turn and right turn boundary of the relative velocity between  $R_i$  and  $O_j$ . In other words, when two robots are approaching each other, they need to either turn left or right sufficiently to avoid each other.

### C. VO Variants

Since VO disregards the reactive nature [18] which enables each robot to independently adapt its velocity to avoid collision with other robots and obstacles, this will lead to unnecessary oscillations when using the VO-based algorithm for distributed multi-robot navigation. Interested readers should refer to [18] for more details. Some variants of VO explicitly consider this problem and make specific improvements to deal with the unnecessary oscillations, such as Reciprocal Velocity Obstacle (RVO) [18] and Hybrid Reciprocal Velocity Obstacle (HRVO) [20]. For RVO, it is assumed that both robots take half the responsibility for collision avoidance, and  $\text{RVO}_{R_i|R_j}(\mathbf{v}_{R_i}, \mathbf{v}_{R_j})$  is defined as follows [18]:

$$\text{RVO}_{R_i|R_j}(\mathbf{v}_{R_i}, \mathbf{v}_{R_j}) = \{\mathbf{v}'_{R_i} | 2\mathbf{v}'_{R_i} - \mathbf{v}_{R_i} \in \text{VO}_{R_i|R_j}(\mathbf{v}_{R_j})\}$$

where  $\mathbf{v}_{R_i}$  and  $\mathbf{v}_{R_j}$  are the current velocity of  $R_i$  and  $R_j$ , and  $\mathbf{v}'_{R_i}$  is the new velocity that will lead to a collision. Each robot chooses a new velocity outside each other's RVO as well as in the same side of each other's RVO, which guarantees collision avoidance. Since obstacles don't have the reactive nature like robots, RVO is unsuitable to realize collision avoidance between a robot and an obstacle. The direction vectors  $\mathbf{vl}$  and  $\mathbf{vr}$  on both sides of RVO is same as VO, and RVO can geometrically be interpreted as VO translated such that its apex lies at  $\frac{\mathbf{v}_{R_i} + \mathbf{v}_{R_j}}{2}$ . HRVO is proposed to solve the oscillations known as "reciprocal dances" when using RVO for distributed multi-robot navigation, for more details refer to [20]. In summary, the difference between VO, RVO and HRVO is the apex with the direction vectors of these three cones being the same.

### D. Obstacle Avoidance Between Polytopes

For the distributed multi-robot navigation with polytopic shapes, it is necessary to evaluate whether collision exists between robots and obstacles, and this can be described as a minimum distance problem between polytopes. Consider two polytopic sets  $\mathcal{S}_1$  and  $\mathcal{S}_2$ , and the distance between these two sets is given by the following primal problem:

$$\text{dist}(\mathcal{S}_1, \mathcal{S}_2) := \min_{y_1, y_2} \{\|y_1 - y_2\|_2 | A_1 y_1 \leq b_1, A_2 y_2 \leq b_2\} \quad (2)$$

where  $\mathcal{S}_1 = \{y_1 \in \mathbb{R}^n | A_1 y_1 \leq b_1\}$  and  $\mathcal{S}_2 = \{y_2 \in \mathbb{R}^n | A_2 y_2 \leq b_2\}$ ,  $A_1$ ,  $b_1$ ,  $A_2$  and  $b_2$  depend on the robot's and obstacle's positions.  $n$  represents the space dimension and is considered as  $n = 2$  in the rest of this letter. Thus, when the polytopes don't collide with each other, the minimum distance between two polytopes is positive, i.e.,  $\text{dist}(\mathcal{S}_1, \mathcal{S}_2) > 0$ . So we can use (2) to check if there is a collision between robots and obstacles with polytopic-shaped.

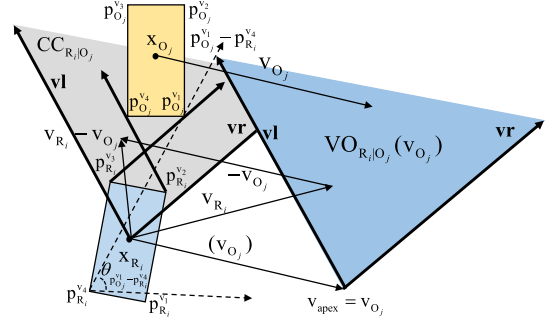


Fig. 3. Velocity obstacle  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$  of robot  $R_i$  induced by the obstacle  $O_j$  for polytopic-shaped robots. First, we need to obtain the direction vectors  $\mathbf{vl}$  and  $\mathbf{vr}$  (bold black solid lines) on both sides of the collision cone  $\text{CC}_{R_i|O_j}$  by connecting the vertices of  $R_i$  and  $O_j$  in any two pairs as done in (3). Then we construct the  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$  with the direction vectors  $\mathbf{vl}$  and  $\mathbf{vr}$ , and  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$  is a cone with its apex at  $\mathbf{v}_{O_j}$ .

## III. MULTI-ROBOT NAVIGATION WITH POLYTOPIC-SHAPED ROBOTS

In this section, we firstly propose an optimization-free approach to construct the VO for polytopic-shaped robots in Section III-A. Then, we demonstrate how to realize the distributed multi-robot navigation with polytopic shapes using VO and its variants in Section III-B.

### A. Velocity Obstacle for Polytopic-Shaped Robots

In Section II-B, we have reviewed the concept of VO and demonstrated how to construct the VO for circular-shaped robots. In the following, we will extend the concept of VO to polytopic-shaped robots and obstacles and demonstrate how to construct the VO for polytopic-shaped robots with high computational efficiency.

In order to construct VO for polytopic-shaped robots, we are going to use the vertex coordinates, together with the robot's other external states, including robot's current position and velocity. Assume there is a robot  $R_i$  with  $K_{R_i}$  vertices and an obstacle  $O_j$  with  $K_{O_j}$  vertices in the shared environment, and the current position, current velocity, and each vertex's current coordinate of  $R_i$  and  $O_j$  could be denoted as  $\mathbf{x}_{R_i}$ ,  $\mathbf{x}_{O_j}$ ,  $\mathbf{v}_{R_i}$ ,  $\mathbf{v}_{O_j}$ ,  $\mathbf{p}_{R_i}^{v_k}$ ,  $\mathbf{p}_{O_j}^{v_h}$ , respectively. As we mentioned in Section II-B, calculating the direction vectors  $\mathbf{vl}$  and  $\mathbf{vr}$  on both side of the cone  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$  is necessary and sufficient to construct  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$ , and in the following we will demonstrate how to calculate it.

For two polytopes  $R_i$  and  $O_j$ , a vertex pair  $(\mathbf{p}_{O_j}^{v_h}, \mathbf{p}_{R_i}^{v_k})$  could be obtained by selecting any two vertices of  $O_j$  and  $R_i$ , where  $h \in \{1, 2, \dots, K_{O_j}\}$  and  $k \in \{1, 2, \dots, K_{R_i}\}$ , and there are  $K_{R_i} \times K_{O_j}$  vertex pairs in total. By connecting the two vertices in a vertex pair  $(\mathbf{p}_{O_j}^{v_h}, \mathbf{p}_{R_i}^{v_k})$  as a vector, we could obtain the angle between the vector  $\mathbf{p}_{O_j}^{v_h} - \mathbf{p}_{R_i}^{v_k}$  and the x axis:

$$\theta_{\mathbf{p}_{O_j}^{v_h} - \mathbf{p}_{R_i}^{v_k}} = \tan^{-1}[(\mathbf{p}_{O_j}^{v_h} - \mathbf{p}_{R_i}^{v_k}) \cdot \mathbf{e}_y / (\mathbf{p}_{O_j}^{v_h} - \mathbf{p}_{R_i}^{v_k}) \cdot \mathbf{e}_x] \\ \in [-\pi, \pi]$$

---

**Algorithm 1:** Distributed Navigation for Multi-Robot Systems with VO.

---

**Initialization:** All robots' start positions and goal positions.  
Initialize all robots states and set  $t = 0$ .  
**while**  $t \leq t_{\max}$  **and** at least one robot doesn't arrive at the goal position and is not stopped **do**  
  **for**  $R_i \in \mathbb{N}$  **do**  
    Calculate the new velocity  $\mathbf{v}_{R_i}^{\text{new}}$  for each robot  $R_i$  according (7) and (8).  
  **end for**  
  **for**  $R_i \in \mathbb{N}$  **do**  
    Update the position  $\mathbf{x}_{R_i}$  with robot dynamics.  
  **end for**  
  **for**  $R_i \in \mathbb{N}$  **do**  
    Check the minimum distance with other robots and obstacles (2), and stop robot  $R_i$  if minimum distance is zero.  
  **end for**  
   $t = t + \Delta t$ .  
**end while**

---

as shown in Fig. 3. Then we could obtain the direction vectors  $\mathbf{vl}$  and  $\mathbf{vr}$  on both sides of the cone as follows,

$$\begin{aligned} \mathbf{vl} &= \cos \left( \max_{v_h, v_k} \theta_{\mathbf{p}_{O_j}^{v_h} - \mathbf{p}_{R_i}^{v_k}} \right) \mathbf{e}_x + \sin \left( \max_{v_h, v_k} \theta_{\mathbf{p}_{O_j}^{v_h} - \mathbf{p}_{R_i}^{v_k}} \right) \mathbf{e}_y \\ \mathbf{vr} &= \cos \left( \min_{v_h, v_k} \theta_{\mathbf{p}_{O_j}^{v_h} - \mathbf{p}_{R_i}^{v_k}} \right) \mathbf{e}_x + \sin \left( \min_{v_h, v_k} \theta_{\mathbf{p}_{O_j}^{v_h} - \mathbf{p}_{R_i}^{v_k}} \right) \mathbf{e}_y \end{aligned} \quad (3)$$

where  $\mathbf{vl}$  represents the unit vector corresponding to the largest angle with respect to the x axis among all the vertex pairs, and  $\mathbf{vr}$  represents the smallest one, as shown in Fig. 3.

After calculating the unit vectors  $\mathbf{vl}$  and  $\mathbf{vr}$  on both sides of the cone, and as discussed in Section II-B,  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$  can be then easily constructed as a cone with its apex at  $\mathbf{v}_{O_j}$ , shown in Fig. 3.

### B. Navigation of Distributed Multi-Robot System

This section mainly demonstrates how to achieve the distributed multi-robot navigation with polytopic shapes and the definition of this problem can be found in Section II-A.

For the distributed multi-robot navigation with polytopic shapes, we assume that all robots navigate to their goal positions using the same policy, and the pseudocode of the overall method is shown in Algorithm 1. The most critical part of the whole algorithm is about how to select a new velocity  $\mathbf{v}_{R_i}^{\text{new}}$  for each robot  $R_i$ . In the following we will demonstrate how to select the new velocity  $\mathbf{v}_{R_i}^{\text{new}}$ .

1) *Construct Combined Velocity Obstacle for Each Robot:* We have introduced  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$  to achieve collision avoidance between the robot  $R_i$  and the obstacle  $O_j$  by choosing a velocity  $\mathbf{v}_{R_i}$  which is outside of  $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$ . During the navigation progress, if a robot  $R_i$  needs to avoid collision with all other robots and obstacles around it, it is necessary to select a velocity outside of all the VO induced by each robot or obstacle. Before introducing the combined velocity obstacle [18], we first define the set of neighboring robots and obstacles around the robot  $R_i$  as  $\mathbb{B}_i$  and  $\mathbb{C}_i$ , respectively. The

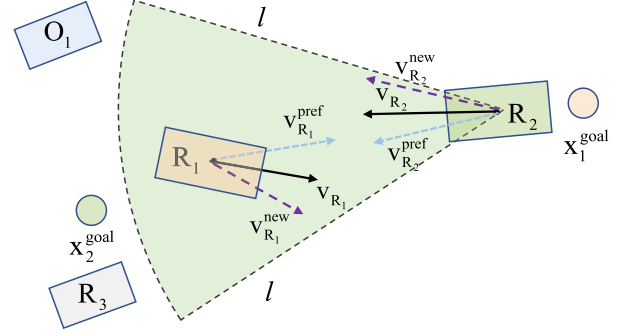


Fig. 4. Illustration for the distributed navigation for multi-robot system. Robot  $R_2$  only considers the robots and obstacles within a certain range ( $l$ ), so  $R_2$  only considers the VO induced by  $R_1$ .

neighboring robots and obstacles around the current position of  $R_i$  within a distance of magnitude  $l$  will be taken into account, and we set  $\mathbb{B}_i = \{j | R_j \in \mathbb{N}, j \neq i, \|\mathbf{x}_{R_i} - \mathbf{x}_{R_j}\|_2 \leq l\}$  and  $\mathbb{C}_i = \{j | O_j \in \mathbb{O}, \|\mathbf{x}_{R_i} - \mathbf{x}_{O_j}\|_2 \leq l\}$ .  $l$  is a variable representing the neighboring region, as shown in Fig. 4, and the optimal  $l$  depends on the maximum velocity of each robot and the obstacle as well as the time  $\tau$ , which represents the time window for collision-free motion. For any robot  $R_i$  with the obstacle  $O_j$ , the optimal neighboring region could be set as (4):

$$l = (v_{R_i, \max} + v_{O_j, \max})\tau \quad (4)$$

Then we define the combined velocity obstacle for robot  $R_i$  as (5):

$$\text{VO}_i = \bigcup_{j \in \mathbb{B}_i} \text{VO}_{R_i|R_j} \cup \bigcup_{j \in \mathbb{C}_i} \text{VO}_{R_i|O_j} \quad (5)$$

which is a union of all the VO induced by each robot or obstacle around the robot  $R_i$ . Therefore, each robot  $R_i$  could realize collision avoidance with other robots and obstacles by choosing a velocity outside of its combined velocity obstacle  $\text{VO}_i$ .

In order to implement RVO and HRVO for polytopic-shaped robots, the combined reciprocal velocity obstacle and the combined hybrid reciprocal velocity obstacle are defined as follows:

$$\begin{aligned} \text{RVO}_i &= \bigcup_{j \in \mathbb{B}_i} \text{RVO}_{R_i|R_j} \cup \bigcup_{j \in \mathbb{C}_i} \text{VO}_{R_i|O_j} \text{HRVO}_i \\ &= \bigcup_{j \in \mathbb{B}_i} \text{HRVO}_{R_i|R_j} \cup \bigcup_{j \in \mathbb{C}_i} \text{VO}_{R_i|O_j} \end{aligned} \quad (6)$$

where VO induced by the other robots are replaced by RVO or HRVO to reduce unnecessary oscillation.

2) *Selecting Velocity for Each Robot:* To realize the distributed multi-robot navigation with polytopic shapes, the selected velocity not only needs to avoid collision with other robots and obstacles but also needs to guide the robot to its goal position. So we could choose a new velocity  $\mathbf{v}_{R_i}^{\text{new}}$  that is outside of the combined velocity obstacle  $\text{VO}_i$  and as close as possible to the preferred velocity for each robot  $R_i$  independently as follows,

$$\begin{aligned} \mathbf{v}_{R_i}^{\text{new}} &= \underset{\mathbf{v}_{R_i} \notin \text{VO}_i}{\text{argmin}} \|\mathbf{v}_{R_i} - \mathbf{v}_{R_i}^{\text{pref}}\|_2, \mathbf{v}_{R_i}^{\text{pref}} \\ &= v_{R_i, \max} \frac{\mathbf{x}_{R_i} - \mathbf{x}_i^{\text{goal}}}{\|\mathbf{x}_{R_i} - \mathbf{x}_i^{\text{goal}}\|_2} \end{aligned} \quad (7)$$

where the direction of the preferred velocity  $\mathbf{v}_{R_i}^{\text{pref}}$  is from the current position  $\mathbf{x}_{R_i}$  to the goal position  $\mathbf{x}_i^{\text{goal}}$  to guide the robot to its goal position as fast as possible, shown in Fig. 4.

*Remark 2:* When the environment is crowded with many static obstacles, selecting a preferred velocity as above may result in deadlock, which could be improved by a global path planner, such as RRT\* [26]. Notice that the global path planner configuration is out of scope of this letter.

However, there may be no feasible solution for (7) in extremely crowded environment, where the combined velocity obstacle  $\text{VO}_i$  saturates the entire velocity space and picking a velocity outside  $\text{VO}_i$  is impossible. Moreover, choosing any velocity in this case will result in a collision, only the expected collision time is different. Therefore we need to make a trade-off between safety maneuver and traveling to the goal position as quickly as possible. We can select the new velocity  $\mathbf{v}_{R_i}^{\text{new}}$  inside the combined velocity obstacle  $\text{VO}_i$  as follows,

$$\mathbf{v}_{R_i}^{\text{new}} = \underset{\mathbf{v}_{R_i} \in \text{VO}_i}{\text{argmin}} J(\mathbf{v}_{R_i}), J(\mathbf{v}_{R_i}) = \phi_i \frac{1}{tc_i(\mathbf{v}_{R_i})} + \|\mathbf{v}_{R_i} - \mathbf{v}_{R_i}^{\text{pref}}\|_2 \quad (8)$$

where  $J(\mathbf{v}_{R_i})$  is a penalty cost function for velocity, the penalty of the candidate velocity depends on the expected collision time  $tc_i(\mathbf{v}_{R_i})$  and the difference between the candidate velocity and the preferred velocity  $\mathbf{v}_{R_i}^{\text{pref}}$ ,  $\phi_i$  is the weight coefficient between these two components for each robot  $R_i$ , and we choose the velocity with the minimum penalty cost as the new velocity  $\mathbf{v}_{R_i}^{\text{new}}$ . For the expected collision time  $tc_i(\mathbf{v}_{R_i})$ , we calculate it as follows,

$$tc_i(\mathbf{v}_{R_i}) = \min_{R_j \in \mathbb{B}_i, O_j \in \mathbb{C}_i} \left\{ tc_{R_i}^{R_j}(\mathbf{v}_{R_i}), tc_{R_i}^{O_j}(\mathbf{v}_{R_i}) \right\} \quad (9)$$

where  $tc_{R_i}^{R_j}$  or  $tc_{R_i}^{O_j}$  is the expected collision time estimated from the current position and the velocity of the robot  $R_i$  for collision with robot  $R_j$  or obstacle  $O_j$ . If there is no collision, the expected collision time will be set as infinity.

#### IV. SIMULATION RESULTS

##### A. Simulation Setup

We use an Ubuntu Laptop with Intel Core i7-9750H (CPU 4.5 GHz), NVIDIA GTX 1650 (GPU, 1665 MHz) and the whole simulation environment is setup with Python for all computations. The size of the simulation scene is  $10 \text{ m} \times 10 \text{ m}$ , the time step  $\Delta t$  of simulation is set as 0.1 s, and the maximum time  $t_{\text{max}}$  of loop is set as 30 s. In our simulation trails, all the robots are non-holonomic and controlled by the transitional speed  $v$  and angular speed  $\omega$ . However, the new velocity  $\mathbf{v}_{R_i}^{\text{new}}$  selected by the VO-based navigation algorithm is the orthogonal velocity  $\mathbf{v}_{R_i}^{\text{new}} = [\mathbf{v}_x^{\text{new}}, \mathbf{v}_y^{\text{new}}]$ . To convert to a form which can be used by non-holonomic robots, the orthogonal velocity  $\mathbf{v}_{R_i}^{\text{new}}$  is converted to transitional speed  $v$  and angular speed  $\omega$  as follows:

$$v = \|\mathbf{v}_{R_i}^{\text{new}}\| \cdot \cos \varsigma, \quad \omega = -\varsigma/\eta$$

where  $\varsigma$  is the orientation difference between the robot's current orientation and the direction of the orthogonal velocity  $\mathbf{v}_{R_i}^{\text{new}}$ , and  $\eta$  is the time to adjust the orientation difference to 0. The simulation parameters associated with the robot are listed in Table II.

TABLE II  
SETUP OF THE SIMULATION PARAMETER ASSOCIATED WITH ROBOT

| Notation              | Meaning                                    | Value     |
|-----------------------|--------------------------------------------|-----------|
| $v_{R_i, \text{max}}$ | Robot's maximum orthogonal velocity        | 1.5 m/s   |
| $v_{\text{max}}$      | Robot's maximum transitional speed         | 1.5 m/s   |
| $w_{\text{max}}$      | Robot's maximum angular speed              | 1.0 rad/s |
| $\eta$                | Time of adjusting orientation difference   | 0.2 s     |
| $l$                   | Robot's neighboring region                 | 5.0 m     |
| $\phi$                | Weight coefficient of the penalty function | 4.0       |

Moreover, a safe margin of 0.15 m is added to each polytopic robot to ensure tolerance for motion integration errors and to guarantee a minimum safety distance.

To distinguish with the circular-shaped based VO and for the sake of simplicity, in the following part we denote circular-shaped based VO as  $\text{VO}_c$  [4], our polytopic-shaped based VO as  $\text{VO}_p$ , circular-shaped based RVO as  $\text{RVO}_c$  [18], our polytopic-shaped based RVO as  $\text{RVO}_p$ , circular-shaped based HRVO as  $\text{HRVO}_c$  [20], and our polytopic-shaped based HRVO as  $\text{HRVO}_p$ . In the following, we will validate the performance of our approach for distributed multi-robot navigation with polytopic shapes in many challenging scenarios.

##### B. Navigation of Distributed Multi-Robot System in Circle Scenario

We first validate the approach for distributed multi-robot navigation with polytopic shapes in the circle scenario: a certain number of polytopic-shaped robots are uniformly located on a circle of radius 4 m centered at (5 m, 5 m), and the initial and goal position of robots are symmetric along the center of the circle, as shown in Fig. 5(a). Our approach for polytopic obstacle avoidance with  $\text{VO}_p$ , together with its variants on  $\text{RVO}_p$  and  $\text{HRVO}_p$ , are analyzed in the numerical simulation trails. The simulation results of the navigation task for eight heterogeneous polytopic-shaped robots with a circular configuration are shown in Fig. 5, where all our navigation algorithms move each robot to its goal position while avoiding collision with other robots. The trajectories generated by  $\text{VO}_p$ ,  $\text{RVO}_p$  and  $\text{HRVO}_p$  are also illustrated in Fig. 5, where  $\text{RVO}_p$  and  $\text{HRVO}_p$  have fewer unnecessary oscillations compared with  $\text{VO}_p$ . Moreover, our approach could achieve the distributed multi-robot navigation with polytopic shapes in a large-scaled multi-robot systems with up to 16 robots, for more details readers can refer to the attached video.

##### C. Navigation With Static and Dynamic Obstacles

We also validate the approach for the distributed multi-robot navigation with polytopic shapes in the scenario with static and dynamic polytopic-shaped obstacles. In this scenario, the initial and goal position of each robot are manually set. Fig. 6 shows the navigation simulation results for 5 polytopic-shaped robots with 2 static and 2 dynamic obstacles in the shared environment. The dynamic obstacles travel at a speed of 1.0 m/s: one dynamic obstacle travels from (8.85 m, 2.35 m) to (5.0 m, 2.35 m) and the other travels from (4.4 m, 4.75 m) to (8.0 m, 4.75 m). According to Fig. 6, we observe that each robot successfully navigates to its goal position while avoiding collision with other robots and obstacles using the navigation algorithm based on  $\text{RVO}_p$ . As shown in Fig. 6(a) and Fig. 6(b), the robots  $R_0$ ,  $R_1$  and  $R_2$  explicitly adjust their velocities to avoid collisions with

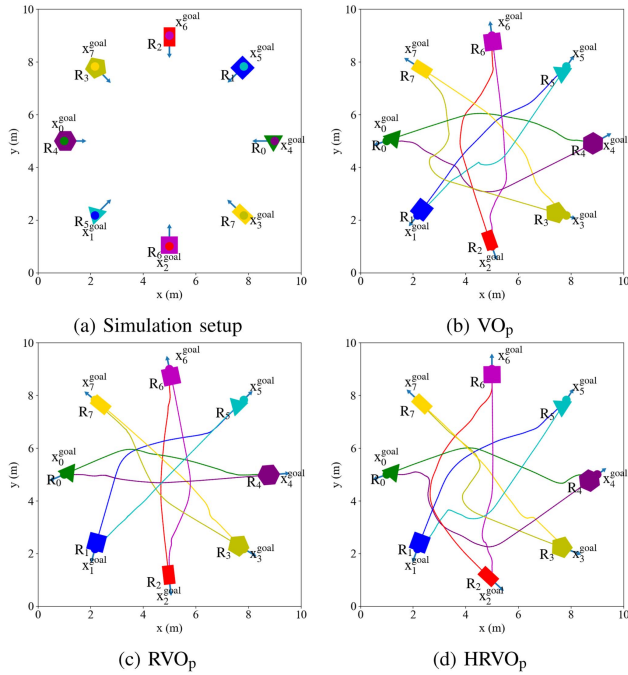


Fig. 5. Simulation results of eight polytopic-shaped robots in a circle scenario using our proposed approach under (b) polytopic-shaped based velocity obstacle (VO<sub>p</sub>), (c) polytopic-shaped based reciprocal velocity obstacle (RVO<sub>p</sub>), (d) polytopic-shaped based hybrid reciprocal velocity obstacle (HRVO<sub>p</sub>) representations. In (a), we show the initial and goal position of each robot in a circle scenario, where the goal position of each robot is shown as a circle with different colors.

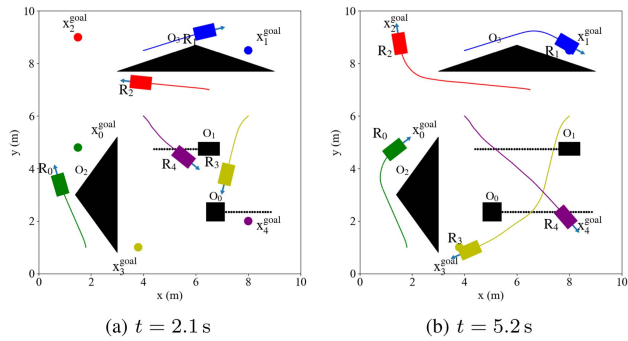


Fig. 6. The navigation simulation results for 5 polytopic-shaped robots with 2 static and 2 dynamic obstacles at two different moments, with the algorithm based on RVO<sub>p</sub>. The black polytopes represent obstacles, and the traveled trajectories of dynamic obstacles are marked as dotted lines.

the static obstacles, and R<sub>3</sub> and R<sub>4</sub> explicitly adjust their velocity to avoid collisions with the dynamic obstacles and other robots.

#### D. Performance Evaluation With Random Scenarios

Since each polytope could be encircled by a hyper-ellipse, we could also use the circular-shaped based VO to realize the distributed multi-robot navigation with polytopic shapes. In each random scenario, we select 8 rectangle robots with dimensions 1.0 m × 0.6 m as simulation objects, and the initial and goal position of each robot are randomly set on a circle of radius 4 m centered at (5 m, 5 m).

Three metrics are utilized to evaluate the method's performance: completion rate, deadlock rate, and average travel distance. The completion rate is the ratio of robots successfully navigating to their goal position without any collisions or deadlock, which describes the method's performance of collision avoidance and navigation. The deadlock rate is the ratio of cases being stuck somewhere during the navigation without any collisions, which describes the method's performance of navigation. Furthermore, the average travel distance refers to the average distance traveled by the robot from the initial position to the goal position for all completed cases, which describes the optimality of the method. All methods (VO<sub>p</sub>, VO<sub>c</sub>, RVO<sub>p</sub>, RVO<sub>c</sub>, HRVO<sub>p</sub>, HRVO<sub>c</sub>) are performed for 100 trails in the random scenario. Additionally, we also resize the robots lengths and widths with ratios varying from 0.4 to 1.4, e.g., a ratio of 0.4 means scaling the robot to 0.4 times of its original size, i.e., 0.4 m × 0.24 m. For these random simulations, we compare the above metrics for different methods and observe how these metrics change with the robot dimension ratio. We list the results in Table III. Intuitively, as the size of robot gets bigger, the completion rate decreases, the deadlock rate and the average travel distance increase. We notice that when the robot dimension ratio is large (>1.0), a conservative approximation of robot shape, i.e., considering polytopic-shaped robots as hyper-ellipses, is more likely to result in a deadlock, which results in a low completion rate for VO<sub>c</sub>, RVO<sub>c</sub> and HRVO<sub>c</sub>. Additionally, our methods can hold better completion and deadlock rates in this case for navigation tasks. We also need to note that when the robot dimension ratio is very large (1.4), all methods have a low completion rate due to deadlock, since robots tend to choose a slower velocity in the crowded scenarios. Moreover, our methods VO<sub>p</sub>, RVO<sub>p</sub>, HRVO<sub>p</sub> outperform VO<sub>c</sub>, RVO<sub>c</sub> and HRVO<sub>c</sub> in terms of the average travel time regardless of the size of robot, which means our methods are more time-optimal.

**Remark 3:** We notice that compared to VO<sub>c</sub>, RVO<sub>c</sub> and HRVO<sub>c</sub> our methods VO<sub>p</sub>, RVO<sub>p</sub> and HRVO<sub>p</sub> obtain slightly lower completion rates when the dimensional ratio is small ( $\leq 1.0$ ), i.e., the robot's length is less than 1.0 m, the width is less than 0.6 m. This comes from discretization errors in simulation and can be resolved with angle padding on  $\mathbf{v}_l$  and  $\mathbf{v}_r$  (3). We also notice that the average travel distance decreases when the ratio is between 1.0 ~ 1.4, since only the simple cases of randomized ones could be completed by all methods, leading to a short travel distance in a statistical bias.

We also compare VO<sub>p</sub> and VO<sub>c</sub> in the turnaround scenario shown in Fig. 7, the initial and goal position of the robot is (9.0 m, 5.0 m) and (1.0 m, 5.0 m), and the shape of the robot is a rectangle of size 0.8 m × 0.5 m. Due to conservative estimates, the method based on VO<sub>c</sub> causes the robot to bypass the obstacle, leading to a longer path, as shown in Fig. 7(b), while our approach could guide the robot through the narrow corridor to the goal position directly, as shown in Fig. 7(b). Therefore we verify that our approach is more efficient in travel distance than the circular-shaped based VO.

Our work also have some shortcomings. If we select a preferred velocity as in (7) in a crowded environment with many static obstacles, it will result in deadlock or some unsafe scenarios due to the absence of a global planner in this work, as discussed in Remark 2. Furthermore, this work doesn't consider model uncertainties, sensor measurement noise, and real world localization and mapping uncertainties.

TABLE III

A PERFORMANCE COMPARISON OF PROPOSED APPROACHES (POLYTOPIC-SHAPED BASED VO ( $VO_p$ ), RVO ( $RVO_p$ ), HRVO ( $HRVO_p$ )) WITH THE STATE OF THE ART (CIRCULAR-SHAPED BASED VO ( $VO_c$ ) [4], RVO ( $RVO_c$ ) [18], HRVO ( $HRVO_c$ ) [20]) IN THE RANDOM SCENARIO WITH DIFFERENT SIZES OF ROBOTS

| Robot Dimen.<br>Ratio | Completion Rate (%) |            |         |           |          |            | Deadlock Rate (%) |           |         |           |          |           | Average Travel Distance (m) / std |                  |           |                  |           |                  |
|-----------------------|---------------------|------------|---------|-----------|----------|------------|-------------------|-----------|---------|-----------|----------|-----------|-----------------------------------|------------------|-----------|------------------|-----------|------------------|
|                       | $VO_c$              | $VO_p$     | $RVO_c$ | $RVO_p$   | $HRVO_c$ | $HRVO_p$   | $VO_c$            | $VO_p$    | $RVO_c$ | $RVO_p$   | $HRVO_c$ | $HRVO_p$  | $VO_c$                            | $VO_p$           | $RVO_c$   | $RVO_p$          | $HRVO_c$  | $HRVO_p$         |
| 0.4                   | 100                 | 98         | 100     | 98        | 100      | 100        | 0                 | 0         | 0       | 0         | 0        | 0         | 7.53/0.40                         | <b>7.49/0.39</b> | 7.59/0.44 | <b>7.54/0.39</b> | 7.51/0.34 | <b>7.49/0.38</b> |
| 0.6                   | 100                 | <b>100</b> | 100     | 99        | 100      | <b>100</b> | 0                 | <b>0</b>  | 0       | <b>0</b>  | 0        | <b>0</b>  | 7.72/0.74                         | <b>7.64/0.50</b> | 7.83/0.67 | <b>7.74/0.59</b> | 7.87/0.76 | <b>7.64/0.50</b> |
| 0.8                   | 100                 | 95         | 100     | 96        | 100      | 96         | 0                 | <b>0</b>  | 0       | <b>0</b>  | 0        | <b>0</b>  | 8.04/1.00                         | <b>7.91/0.80</b> | 8.05/1.04 | <b>7.74/0.56</b> | 8.17/1.24 | <b>7.76/0.71</b> |
| 1.0                   | 85                  | 77         | 95      | <b>95</b> | 82       | <b>94</b>  | 15                | <b>10</b> | 5       | <b>5</b>  | 13       | <b>5</b>  | 8.44/1.19                         | <b>8.08/1.09</b> | 8.62/1.71 | <b>7.90/0.73</b> | 8.67/1.42 | <b>7.81/0.54</b> |
| 1.1                   | 71                  | <b>73</b>  | 66      | <b>86</b> | 63       | <b>95</b>  | 15                | <b>10</b> | 5       | <b>5</b>  | 15       | <b>5</b>  | 8.08/1.38                         | <b>7.98/0.91</b> | 8.09/0.89 | <b>8.02/1.50</b> | 8.39/2.09 | <b>7.68/0.46</b> |
| 1.2                   | 50                  | <b>62</b>  | 33      | <b>69</b> | 54       | <b>84</b>  | 30                | <b>22</b> | 66      | <b>20</b> | 45       | <b>11</b> | 7.53/0.81                         | <b>7.27/0.21</b> | 7.59/0.97 | <b>7.35/0.19</b> | 7.69/0.93 | <b>7.29/0.21</b> |
| 1.3                   | 42                  | <b>47</b>  | 39      | <b>49</b> | 35       | <b>59</b>  | 41                | <b>30</b> | 59      | <b>36</b> | 64       | <b>18</b> | 7.35/0.21                         | <b>7.22/0.10</b> | 7.35/0.21 | <b>7.31/0.11</b> | 7.35/0.15 | <b>7.28/0.17</b> |
| 1.4                   | 24                  | <b>30</b>  | 32      | <b>38</b> | 25       | <b>42</b>  | 56                | <b>55</b> | 63      | <b>45</b> | 75       | <b>35</b> | 7.44/0.22                         | <b>7.41/0.21</b> | 7.47/0.12 | <b>7.44/0.19</b> | 7.55/0.35 | <b>7.39/0.23</b> |

The bold ones indicate that our approach is superior to the corresponding state of the art.

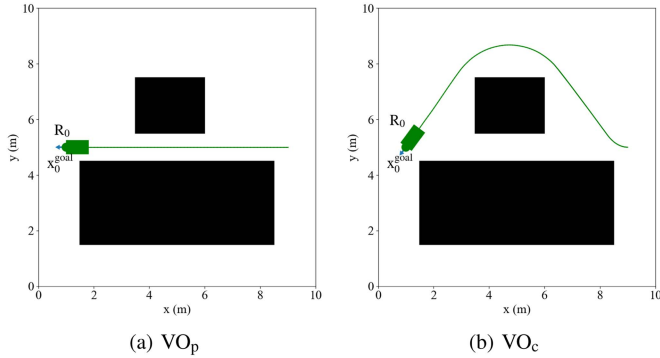


Fig. 7. The comparison of  $VO_p$  and  $VO_c$ , using  $VO_p$  could generate a less-conservative trajectory to pass through the corridor between obstacles instead of moving around them.

## V. CONCLUSION

In this letter, we have proposed a velocity obstacle ( $VO$ )-based approach for distributed multi-robot navigation with polytopic shapes. We first proposed an optimization-free approach to realize the collision avoidance between two polytopic objects by constructing the  $VO$  between them. Then, we proposed a  $VO$ -based approach for distributed multi-robot navigation with polytopic shapes and validated it in many challenging scenarios. Numerical simulation results demonstrated that our approach has good navigation performance and outperforms the state of the art in terms of the completion rate, deadlock rate, and average travel distance. In our future work, we plan to extend our proposed approach to 3D space.

## REFERENCES

- [1] S.-J. Chung, A. A. Paranjape, P. Dames, S. Shen, and V. Kumar, "A survey on aerial swarm robotics," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 837–855, Aug. 2018.
- [2] P. Yu and D. V. Dimarogonas, "Distributed motion coordination for multi-robot systems under LTL specifications," *IEEE Trans. Robot.*, vol. 38, no. 2, pp. 1047–1062, Apr. 2022.
- [3] J. P. Queralta et al., "Collaborative multi-robot search and rescue: Planning, coordination, perception, and active vision," *IEEE Access*, vol. 8, pp. 191617–191643, 2020.
- [4] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *Int. J. Robot. Res.*, vol. 17, no. 7, pp. 760–772, 1998.
- [5] J. Ziegler and C. Stiller, "Fast collision checking for intelligent vehicle motion planning," in *Proc. IEEE Intell. Veh. Symp.*, 2010, pp. 518–522.
- [6] I. E. Grossmann, "Review of nonlinear mixed-integer and disjunctive programming techniques," *Optim. Eng.*, vol. 3, no. 3, pp. 227–252, 2002.
- [7] B. Li and Z. Shao, "A unified motion planning method for parking an autonomous vehicle in the presence of irregularly placed obstacles," *Knowl.-Based Syst.*, vol. 86, pp. 11–20, 2015.
- [8] R. Deits and R. Tedrake, "Efficient mixed-integer planning for UAVs in cluttered environments," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2015, pp. 42–49.
- [9] X. Zhang, A. Liniger, A. Sakai, and F. Borrelli, "Autonomous parking using optimization-based collision avoidance," in *Proc. IEEE Conf. Decis. Control*, 2018, pp. 4327–4332.
- [10] X. Zhang, A. Liniger, and F. Borrelli, "Optimization-based collision avoidance," *IEEE Trans. Control Syst. Technol.*, vol. 29, no. 3, pp. 972–983, May 2021.
- [11] R. Firooz, L. Ferranti, X. Zhang, S. Nejadnik, and F. Borrelli, "A distributed multi-robot coordination algorithm for navigation in tight environments," 2020, *arXiv:2006.11492*.
- [12] A. Thirugnanam, J. Zeng, and K. Sreenath, "Duality-based convex optimization for real-time obstacle avoidance between polytopes with control barrier functions," in *Proc. Amer. Control Conf.*, 2022, pp. 2239–2246.
- [13] A. Thirugnanam, J. Zeng, and K. Sreenath, "Safety-critical control and planning for obstacle avoidance between polytopes with control barrier functions," in *Proc. Int. Conf. Robot. Automat.*, 2022, pp. 286–292.
- [14] P. Glotfelter, J. Cortés, and M. Egerstedt, "Nonsmooth barrier functions with applications to multi-robot systems," *IEEE control Syst. Lett.*, vol. 1, no. 2, pp. 310–315, Oct. 2017.
- [15] J. Zeng, B. Zhang, and K. Sreenath, "Safety-critical model predictive control with discrete-time control barrier function," in *Proc. Amer. Control Conf.*, 2021, pp. 3882–3889.
- [16] J. Asiain and J. Godoy, "Navigation in large groups of robots," *Curr. Robot. Rep.*, vol. 1, no. 4, pp. 203–213, 2020.
- [17] M. Raibail et al., "Decentralized multi-robot collision avoidance: A systematic review from 2015 to 2021," *Symmetry*, vol. 14, no. 3, 2022, Art. no. 610.
- [18] J. v. d. Berg, M. Lin, and D. Manocha, "Reciprocal velocity obstacles for real-time multi-agent navigation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2008, pp. 1928–1935.
- [19] J. v. d. Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics Research*. Berlin, Germany: Springer, 2011, pp. 3–19.
- [20] J. Snape, J. v. d. Berg, S. J. Guy, and D. Manocha, "The hybrid reciprocal velocity obstacle," *IEEE Trans. Robot.*, vol. 27, no. 4, pp. 696–706, Aug. 2011.
- [21] J. Alonso-Mora, A. Breitenmoser, P. Beardsley, and R. Siegwart, "Reciprocal collision avoidance for multiple car-like robots," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2012, pp. 360–366.
- [22] J. Alonso-Mora, A. Breitenmoser, M. Ruffi, P. Beardsley, and R. Siegwart, "Optimal reciprocal collision avoidance for multiple non-holonomic robots," in *Distributed Autonomous Robotic Systems*. Berlin, Germany: Springer, 2013, pp. 203–216.
- [23] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan, "Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2018, pp. 6252–6259.
- [24] H. Li, B. Weng, A. Gupta, J. Pan, and W. Zhang, "Reciprocal collision avoidance for general nonlinear agents using reinforcement learning," 2019, *arXiv:1910.10887*.
- [25] R. Han et al., "Reinforcement learned distributed multi-robot navigation with reciprocal velocity obstacle shaped rewards," *IEEE Robot. Automat. Lett.*, vol. 7, no. 3, pp. 5896–5903, Jul. 2022.
- [26] S. Karaman and E. Frazzoli, "Optimal kinodynamic motion planning using incremental sampling-based methods," in *Proc. IEEE 49th Conf. Decis. Control*, 2010, pp. 7681–7687.